

AutoTestHub 技术栈文档

面试复习专用 · 完整技术栈清单

1. 项目概述

AutoTestHub 是一个 AI 驱动的全栈测试平台，支持接口测试、Web UI 自动化、性能测试三大测试类型。集成 DeepSeek、通义千问、智谱 GLM 等国产大模型，提供 AI 用例生成、RAG 知识库问答、AI 测评师等功能。采用 Django REST Framework + Vue 3 前后端分离架构，Docker Compose 一键部署。

2. 后端技术栈

2.1 Web 框架与 API

技术	版本	用途
Django	4.2.7	Web 框架核心，ORM、中间件、管理后台
Django REST Framework	3.14.0	REST API 构建，ViewSet + Serializer 模式
djangorestframework-simplejwt	5.5.1	JWT 认证 (HS256, Access 24h / Refresh 7d)
django-cors-headers	4.3.0	跨域请求处理
django-filter	23.5	API 查询过滤
Gunicorn	21.2.0	WSGI 生产服务器

2.2 自定义用户模型

继承 Django AbstractUser，新增 role 字段。三级角色体系：admin（管理员，可管理模型配置）、tester（测试工程师，常规操作）、viewer（只读）。所有 API 通过 DRF 权限类进行角色校验。

2.3 异步任务 (Celery)

组件	技术	说明
任务队列	Celery >=5.4.0	异步执行测试任务、AI 评测、知识库向量化
消息代理	Redis >=5.0.0	Celery Broker

组件	技术	说明
结果存储	django-celery-results	任务结果持久化到 Django DB
并发配置	4 workers	每个 worker 独立处理一个任务
超时限制	600 秒	单任务最长执行时间

3. 数据存储

3.1 关系数据库

环境	数据库	说明
生产环境	PostgreSQL 16 (Docker)	完整 SQL 特性，强类型校验
开发环境	SQLite3 (Django 默认)	零配置启动，但需注意约束差异
连接方式	psycopg2-binary 2.9.9	Python PostgreSQL 驱动

注意：SQLite 不会校验 VARCHAR 长度等约束，而 PostgreSQL 会严格校验。开发环境应尽量与生产环境保持一致。

3.2 向量数据库 (Milvus)

组件	技术	说明
向量数据库	Milvus v2.4.0	分布式向量检索，支持亿级向量
Python SDK	pymilvus >=2.4.0	Python 客户端
本地模式	milvus-lite >=2.4.0	嵌入式模式，无需 Docker
元数据存储	etcd v3.5.5	Milvus 集群元数据
对象存储	MinIO	Milvus 数据持久化

3.3 缓存

组件	技术	用途
Redis	7-alpine (Docker)	缓存 + Celery Broker + 会话存储

4. AI/LLM 集成

4.1 LLM Provider 抽象层

采用 工厂模式 + 策略模式 实现统一 LLM 接口：
LLMProviderFactory.create(provider_name) → BaseLLMProvider

所有 Provider 继承同一基类，实现统一接口：
chat(messages) 同步对话
chat_stream(messages) 流式输出 (SSE)
embed(texts) 文本向量化

新增 Provider 只需：1) 继承 BaseLLMProvider 2) 实现三个方法 3) 注册到工厂类

Provider	默认模型	特点
通义千问 (DashScope)	qwen-plus	Chat + Streaming + Embedding (text-embedding-v3)
DeepSeek	deepseek-chat	高性价比，擅长代码生成和推理
智谱 GLM	glm-4-flash	轻量模型，快速响应

4.2 LangChain/LangGraph 生态

包	版本	用途
langchain	>=0.3.0	LLM/Tool 抽象层
langchain-openai	>=0.2.0	OpenAI 兼容 Provider (DeepSeek 等)
langchain-community	>=0.3.0	文档加载器 (PDF/TXT)
langchain-text-splitters	>=0.3.0	文本语义分块
langgraph	>=0.2.0	Agent 状态机编排

4.3 RAG 知识库链路

完整流程：上传文档 → PDF/TXT/DOCX 解析 → 文本语义分块 → Embedding 向量化 → Milvus 向量存储 → 用户提问 → 语义检索相似文档 → 拼接 Prompt → LLM 生成回答

Embedding 方案：默认使用 DashScope text-embedding-v3 (在线)，也支持 sentence-transformers + BAAI/bge-small-zh-v1.5 (本地离线)

4.4 AI 视觉 Web 自动化 (Midscene)

组件	技术	说明
视觉 AI 引擎	Midscene (字节跳动)	AI 视觉驱动的面页元素识别与操作
视觉模型	qwen-vl-plus / qwen-vl-max	多模态大模型理解页面截图
浏览器引擎	Playwright	Chromium/Firefox/WebKit 跨浏览器支持
双引擎模式	Playwright 精确模式 + AI 模式	常规操作用 Playwright，复杂场景用 AI

5. 前端技术栈

技术	版本	用途
Vue 3	^3.5.34	渐进式前端框架，Composition API
Vite	^8.0.12	极速构建工具
Pinia	^3.0.4	状态管理
Vue Router	^5.0.7	SPA 路由
Element Plus	^2.14.0	企业级 UI 组件库
Axios	^1.16.1	HTTP 请求封装，拦截器处理 Token
ECharts	^6.0.0	数据可视化图表
html2pdf.js	^0.14.0	前端 PDF 导出

前端共 19 个路由页面，覆盖六大核心场景：
仪表盘、用例管理（接口/Web/性能）、执行监控、测试报告、AI 模块（测评师/知识库/技能库）、平台管理（模型配置/MCP工具）

6. 测试引擎

引擎	技术	说明
接口测试	pytest 7.4.3 + allure 2.13.2	HTTP API 测试，JSONPath 断言，Allure 报告
Web UI 自动化	Playwright + Midscene AI	双引擎：精确选择器 + AI 视觉识别
性能测试	Locust	支持 5 种测试类型，实时压测监控

7. 部署架构

Docker Compose 一键部署：
PostgreSQL 16 + Redis 7 + etcd + MinIO + Milvus 2.4 + Django(Gunicorn) + Celery + Vue(Nginx)

开发环境：仅启动 PostgreSQL Docker 容器（15432 端口），其余本地运行
生产环境：全部容器化，Nginx 统一入口，环境变量注入配置

8. 模块清单

Django App	功能
accounts	用户系统 (admin/tester/viewer 三级角色)
testcases	接口测试用例管理
web_testcases	Web 自动化测试 (Playwright + AI 双引擎)
performance	性能测试 (Locust 压测引擎)
testsuites	测试套件编排
execution	测试执行引擎
reports	测试报告生成
knowledge_base	RAG 知识库管理 + AI 问答
data_factory	测试数据工厂 (AI 生成 + 版本管理)
quality_checker	质量数字人 (用例质量检测)
ai_evaluator	AI 测评师 (知识库机器人/API 评测)
agent_gateway	Agent 网关 (统一 AI 入口, Prompt 管理)

9. 核心设计模式

9.1 工厂模式 — LLM Provider

LLMProviderFactory.create('deepseek') 根据名称返回对应 Provider
实例。新增模型供应商只需注册，无需改动业务代码。

9.2 策略模式 — 测试执行引擎

抽象 TestExecutor 基线，API/Web/Performance 三种执行器各自实现。调度层统一调用 execute() 方法，不关心具体类型。

9.3 观察者模式 — Celery 任务回调

测试任务异步执行完成后，通过回调更新任务状态和数据。前端通过轮询获取实时进度。

`django-celery-results`